

Topic 3.2

3.2: Examples: Fundamentals of Classes

The account example that follows demonstrates fundamentals of classes, including class declaration, constructors, name overloading, and public and private scope.

```
// FILENAME:  demoacct.cpp
// PROGRAMMER: Gary J. Blair
// DATE:      4/11/1996
// COMPILER:  MS Visual C++ 2008 Express Edition
// REQUIRED:   demoacct.cpp, account.h, account.cpp
// PURPOSE:
//   Test account objects.
//   Demonstrate fundamental concepts about classes.

#include <iostream>
#include <stdlib.h>
#include "account.h"

using namespace std;

int main(void)
{
    Account gary(3819,1000.00); // 2-arg constructor
                               // invoked.
    Account dummy;             // Default constructor invoked
    Account no_name             // Default constructor invoked

    gary.withdraw(100.00);
    cout << "After withdrawing $100.00 Gary has $"
         << gary.balance() << endl;

    gary.deposit(200.00);
    cout << "After depositing  $200.00 Gary has $"
         << gary.balance() << endl;

    cout << "Enter information for a new account ";
    no_name.read();
    cout << "The information entered is ";
    no_name.write();
    cout << endl;

    no_name.deposit(12);
    cout <<
        "After depositing $12.00 that account has $"
        << no_name.balance() << endl;

    dummy.deposit(50.00); // Causes error due to
                          // uninitialized account

    return 0;
}
```

Output from this program on a DOS PC:

```
C:\DATA\TEACHING\UCB\CPP> demoacct
After withdrawing $100.00 Gary has $900
After depositing  $200.00 Gary has $1100
Enter information for a new account 1234:169
The information entered is 1234:169
```

After depositing \$12.00 that account has \$181
 *** Attempt to use uninitialized account.

The following program demonstrates fundamental concepts about classes.

```
// FILENAME:   account.h
// PROGRAMMER: Gary J. Blair
// DATE:       4/11/1996
// COMPILER:   MS Visual C++ 2008 Express Edition
// REQUIRED:    demoacct.cpp, account.h, account.cpp
// PURPOSE:
//   Declaration of account objects.
//   Demonstrate fundamental concepts about classes.

class   Account
{
    public: // The following members may be
           // accessed anywhere.

           // Define some representational types.

    typedef   unsigned long AccountNumber;
    typedef   double       Money;

    // The 0-argument constructor.

    Account(void);

    // A 2-argument constructor.

    Account( AccountNumber acct_num,
             Money acct_bal );

    // A member function to deposit money
    // in an account.

    void   deposit( Money amount );
    // A member function to withdraw money
    // from an account.

    void   withdraw( Money amount );

    // A member function to inquire
    // the current balance.

    Money   balance(void);

    // A member function to read account
    // information in the form
    // acct-num:acct-bal from cin.

    void   read(void);

    // A member function to write
    // account information in the
    // form acct-num:acct-bal to cout.

    void   write(void);

    private: // The following members may only
            // be accessed by member functions.

    AccountNumber account_number; // The account
                                // number.
    Money          account_balance; // Account
                                // balance.
}
```

```
int            initialized;    // Has account
                                // been initd?

// A member function to ensure that an
// account has been initialized prior to
// actual use.

void verify_initialization(void);
};

// FILENAME:   account.cpp
// PROGRAMMER: Gary J. Blair
// DATE:       4/11/1996
// COMPILER:   MS Visual C++ 2008 Express Edition
// REQUIRED:    demoacct.cpp, account.h, account.cpp
// PURPOSE:
//   Implementation of account objects.
//   Demonstrate fundamental concepts
//   about classes.

#include <iostream>
#include <stdlib.h>
#include "account.h"

using namespace std;

Account::Account(void)
{
    this->initialized = 0;
}

Account::Account( AccountNumber acct_num,
                  Money acct_bal )
{
    this->initialized = 1;
    this->account_number = acct_num;
    this->account_balance = acct_bal;
}

void Account::deposit( Money amount )
{
    this->verify_initialization();
    this->account_balance += amount;
}

/* The use of this in all above function
   definitions is unnecessary. The compiler
   will provide it when we use an
   unqualified member name in the body of a
   member function of that class. It is never
   wrong to use this, but it is not always
   necessary. There are times when we use this
   to refer to the entire object on which the
   member function is invoked. */

void Account::withdraw( Money amount )
{
    verify_initialization();
    account_balance -= amount;
}

Account::Money Account::balance(void)
{
    verify_initialization();
    return account_balance;
}
```

```
}

void Account::write(void)
{
    verify_initialization();
    cout << account_number << ":"
         << account_balance;
}

void Account::read(void)
{
    cin >> account_number;
    if (cin.get() != ':')
    {
        cerr << "*** Error reading account:
               no colon.\n";
        exit(1);
    }
    cin >> account_balance;
    initialized = 1;
}

void Account::verify_initialization(void)
{
    if (initialized)
        return;
    else
    {
        cerr << "*** Attempt to use uninitialized
               account.\n";
        exit(1);
    }
}
```

Topic 3.3 (<https://onlinelearning.berkeley.edu/courses/2291660/pages/topic-3-dot-3>)